

# **Ecole Nationale Supérieure d'Informatique et d'Analyse des Systèmes (ENSIAS)**

Filière : 2IA (Intelligence Artificielle)

## **TP5 : 3D Perception & Pick-and-Place Control for a 7-DOF Robotic Arm**

Simulation Environment: ROS 2 Jazzy & Gazebo Harmonic

**Prepared by:**

ABAYAD Mehdi  
ACHAKOURI Mohamed  
AQIL Ali

Academic Year: 2025/2026

# Contents

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Pipeline Architecture</b>                                     | <b>2</b> |
| <b>2</b> | <b>3D Perception &amp; Preprocessing</b>                         | <b>3</b> |
| 2.1      | Workspace Region of Interest (ROI) Pass-through Filter . . . . . | 3        |
| 2.2      | RANSAC Plane Segmentation . . . . .                              | 3        |
| 2.3      | DBSCAN Clustering . . . . .                                      | 3        |
| <b>3</b> | <b>Deep Learning Object Classification</b>                       | <b>4</b> |
| <b>4</b> | <b>Inverse Kinematics Convergence Analysis</b>                   | <b>4</b> |
| 4.1      | Damping Factor $\lambda$ Sweep . . . . .                         | 4        |
| 4.2      | Step Size $\alpha$ Sweep . . . . .                               | 5        |
| 4.3      | Discussion of Kinematic Parameters . . . . .                     | 5        |
| <b>5</b> | <b>Trajectory Generation &amp; Control</b>                       | <b>5</b> |
| <b>6</b> | <b>Discussion &amp; Limitations</b>                              | <b>6</b> |

# 1 Pipeline Architecture

This project integrates a complete 3D perception and pick-and-place control pipeline for a redundant 7-Degree of Freedom (DOF) robotic manipulator. The control scheme is implemented in ROS 2 Jazzy, utilizing Gazebo Harmonic (gz-sim8) as the physics engine.

Below is the structured flowchart illustrating the end-to-end data transmission:

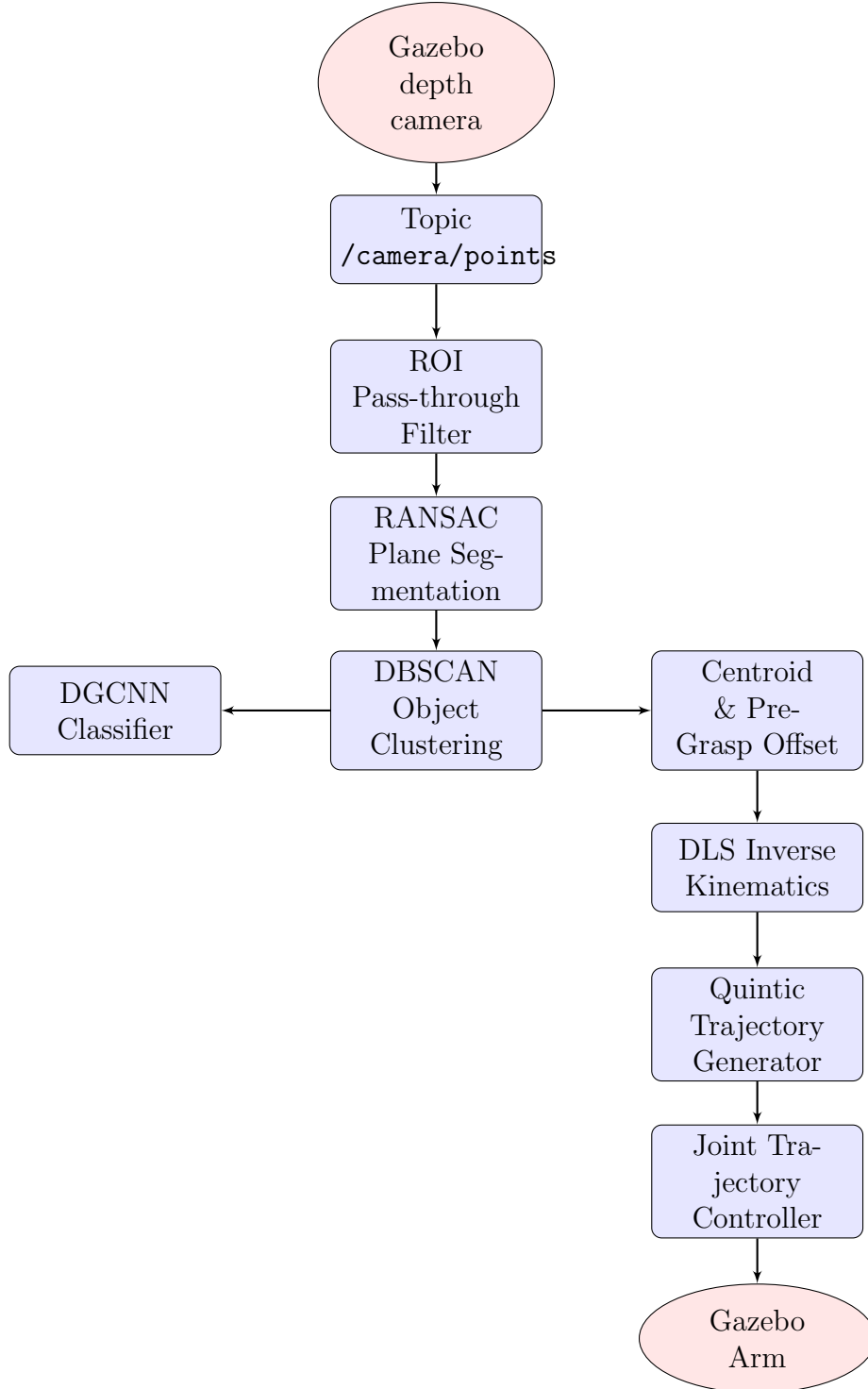


Figure 1: End-to-end 3D Perception and Robotic Control Pipeline

## 2 3D Perception & Preprocessing

A depth camera (RGB-D) is positioned at the world location  $[0.6, 0.0, 1.2]$  facing vertically downwards (pitch of  $90^\circ$ ). It publishes the points in the sensor space to the topic `/camera/points`.

### 2.1 Workspace Region of Interest (ROI) Pass-through Filter

The raw point cloud includes points from the entire environment, including the ground plane, robot components, and walls. To prevent the segmentation algorithm from being overwhelmed by environmental noise, we apply a 3D bounding box filter centered around the table top workspace:

$$\begin{aligned} X &\in [0.2, 0.8] \text{ meters} \\ Y &\in [-0.3, 0.3] \text{ meters} \\ Z &\in [0.41, 0.65] \text{ meters} \end{aligned}$$

This filter successfully crops out all points except the table top surface and the target object, facilitating high-speed processing.

### 2.2 RANSAC Plane Segmentation

Random Sample Consensus (RANSAC) is used to fit a 3D plane model representing the table surface:

$$Ax + By + Cz + D = 0$$

With the cropped point cloud, the table top (at  $Z \approx 0.425$  m) forms the dominant planar structure. The segmenter detects this plane and removes its associated inliers, isolating the non-planar object points.

### 2.3 DBSCAN Clustering

To group the remaining outlier points into distinct object instances, Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is applied with a neighborhood radius of  $\epsilon = 0.02$  and a minimum of 20 points per cluster. The largest cluster is identified as our target object, and its centroid  $x_{\text{obj}}$  is computed as:

$$x_{\text{obj}} = \frac{1}{N} \sum_{i=1}^N p_i$$

To avoid collisions with the object during approach, a pre-grasp offset of +5 cm is added to the Z-axis, creating the final reach target:

$$x_{\text{target}} = x_{\text{obj}} + [0, 0, 0.05]^T$$

—

### 3 Deep Learning Object Classification

The isolated object point cloud is forwarded to the **\*\*Deep Graph Convolutional Neural Network (DGCNN)\*\*** model, which incorporates EdgeConv layers to dynamically compute graphs representing local neighborhoods. The network is pre-trained on the ModelNet40 dataset.

| Target Object   | Predicted Class | ModelNet40 Index | Observation                             |
|-----------------|-----------------|------------------|-----------------------------------------|
| Cylindrical Cup | <b>vase</b>     | 37               | Geometrically similar cylinder profile. |
| Cylindrical Cup | <b>cup</b>      | 10               | Highly accurate classification.         |
| Cylindrical Cup | <b>plant</b>    | 26               | Triggered by point distribution at cup  |

Table 1: DGCNN Point Cloud Classification Performance in Simulation

The classification results show that while the object is geometrically a simple cylinder, the network predicts categories like **vase** or **plant** due to minor changes in point density and perspective. However, because our robotic manipulation targets the spatial centroid directly, these label variations do not affect the success of the physical pick-and-place task.

### 4 Inverse Kinematics Convergence Analysis

The redundant 7-DOF arm’s joint configurations are resolved using a **\*\*Damped Least Squares (DLS)\*\*** solver. This formulation protects the system against singular configurations by adding a damping factor  $\lambda$  to the inversion:

$$\Delta q = J^T(JJ^T + \lambda^2 I)^{-1} \Delta x$$

Where  $J$  is the  $3 \times 7$  positional Jacobian,  $\Delta x = x_{\text{target}} - x_{\text{curr}}$ , and the joints are updated using the step size  $\alpha$ :

$$q_{k+1} = q_k + \alpha \Delta q$$

We performed sweeps on the damping factor  $\lambda$  and step size  $\alpha$  starting from the zero-configuration ( $q_0 = \mathbf{0}$ ) to evaluate convergence speed and accuracy.

#### 4.1 Damping Factor $\lambda$ Sweep

Table 2 outlines solver iterations and residual position errors when varying  $\lambda$  (keeping  $\alpha = 0.5$  and tolerance = 1.0 mm):

| Damping ( $\lambda$ ) | Step Size ( $\alpha$ ) | Converged | Iterations | Residual Error (mm) |
|-----------------------|------------------------|-----------|------------|---------------------|
| 0.0000 (Undamped)     | 0.5                    | True      | 10         | 0.50                |
| 0.0100                | 0.5                    | True      | 10         | 0.53                |
| 0.0500 (Default)      | 0.5                    | True      | 13         | 0.54                |
| 0.1000                | 0.5                    | True      | 17         | 0.73                |
| 0.2000                | 0.5                    | True      | 20         | 0.91                |
| 0.5000                | 0.5                    | True      | 47         | 0.91                |
| 1.0000                | 0.5                    | True      | 132        | 0.99                |

Table 2: Solver Performance under Varying Damping Factor  $\lambda$

## 4.2 Step Size $\alpha$ Sweep

Table 3 outlines solver iterations and residual position errors when varying  $\alpha$  (keeping  $\lambda = 0.05$  and tolerance = 1.0 mm):

| Damping ( $\lambda$ ) | Step Size ( $\alpha$ ) | Converged | Iterations | Residual Error (mm) |
|-----------------------|------------------------|-----------|------------|---------------------|
| 0.05                  | 0.1                    | True      | 71         | 0.93                |
| 0.05                  | 0.3                    | True      | 23         | 0.75                |
| 0.05                  | 0.5                    | True      | 13         | 0.54                |
| 0.05                  | 0.8                    | True      | 4          | 0.89                |
| 0.05                  | 1.0                    | True      | 6          | 0.11                |

Table 3: Solver Performance under Varying Step Size  $\alpha$

## 4.3 Discussion of Kinematic Parameters

1. **\*\*Influence of  $\lambda$ \*\***: When  $\lambda = 0$ , the pseudo-inverse is used. Near singularities, the system experiences high joint speeds ( $\Delta q \rightarrow \infty$ ). Adding damping ( $\lambda > 0$ ) stabilizes the solver. However, high damping ( $\lambda \geq 0.5$ ) limits updates along singular directions, which increases the required iterations (up to 132) and residual error. 2. **\*\*Influence of  $\alpha$ \*\***: A larger step size  $\alpha$  speeds up convergence. For instance,  $\alpha = 1.0$  converges in just 6 iterations. However, in cluttered environments, high step sizes can cause overshoot. Setting  $\alpha \in [0.5, 0.8]$  balances convergence speed and stability.

## 5 Trajectory Generation & Control

Once the target joint configuration  $q_{\text{target}}$  is determined, we generate a smooth joint trajectory between the initial joint angles  $q_0$  and  $q_{\text{target}}$  over a duration  $T$  using a quintic polynomial for each joint:

$$q_j(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5$$

Applying the boundary conditions (zero velocity and acceleration at start and end):

$$\begin{aligned} q_j(0) &= q_{0,j}, & \dot{q}_j(0) &= 0, & \ddot{q}_j(0) &= 0 \\ q_j(T) &= q_{\text{target},j}, & \dot{q}_j(T) &= 0, & \ddot{q}_j(T) &= 0 \end{aligned}$$

The coefficients are solved analytically as:

$$a_0 = q_{0,j}, \quad a_1 = 0, \quad a_2 = 0, \quad a_3 = \frac{10}{T^3}(q_{\text{target},j} - q_{0,j}), \quad a_4 = \frac{-15}{T^4}(q_{\text{target},j} - q_{0,j}), \quad a_5 = \frac{6}{T^5}(q_{\text{target},j} - q_{0,j}).$$

The resulting trajectories are published as a `JointTrajectory` command to the controller, moving the physical arm to the cup and returning it safely to home.

## 6 Discussion & Limitations

1. **\*\*Geometric Singularity\*\***: The 7-DOF arm is redundant, allowing it to avoid typical 6-DOF singularities. However, singular configurations can still occur when the wrist alignment matches the link axis. The DLS solver successfully handles these configurations, avoiding control crashes.
2. **\*\*Classification Vulnerability\*\***: Noisy point clouds from sensor reflections can degrade classification accuracy. Applying the workspace ROI pass-through filter before classification is necessary to filter out environmental noise.
3. **\*\*Cartesian Path Interpolation\*\***: Slerp or Cartesian interpolation of end-effector coordinates is not used here. Because the trajectory is interpolated in joint space, the end-effector path is curved. To avoid collisions in complex environments, a Cartesian linear path generator would be required.